



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 11

Dr. Asad Mahmood

Feb. 10, 2026

Lecture Outline

- Announcements (Assignment and Quiz 2 Next week – Chapter 2)
- Outline of Previous Lecture:
 - **Chapter 2 - Instructions: Language of the Computer**
 - 2.7 Instructions for Making Decisions
 - 2.8 Supporting Procedures in Computer Hardware
- Outline to Today's Lecture:
 - **Chapter 2 - Instructions: Language of the Computer**
 - 2.8 Supporting Procedures in Computer Hardware

Chapter 2

Instructions: Language of the Computer

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - **2.7 Instructions for Making Decisions**
 - 2.8 Supporting Procedures in Computer Hardware
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

Instructions for making decisions

- What distinguishes a computer from a simple calculator is its **ability to make decisions**.
- Otherwise, continue sequentially
- **beq rs1, rs2, L1**
 - if (rs1 == rs2) branch to instruction labeled L1
- **bne rs1, rs2, L1**
 - if (rs1 != rs2) branch to instruction labeled L1

Conditional Operations

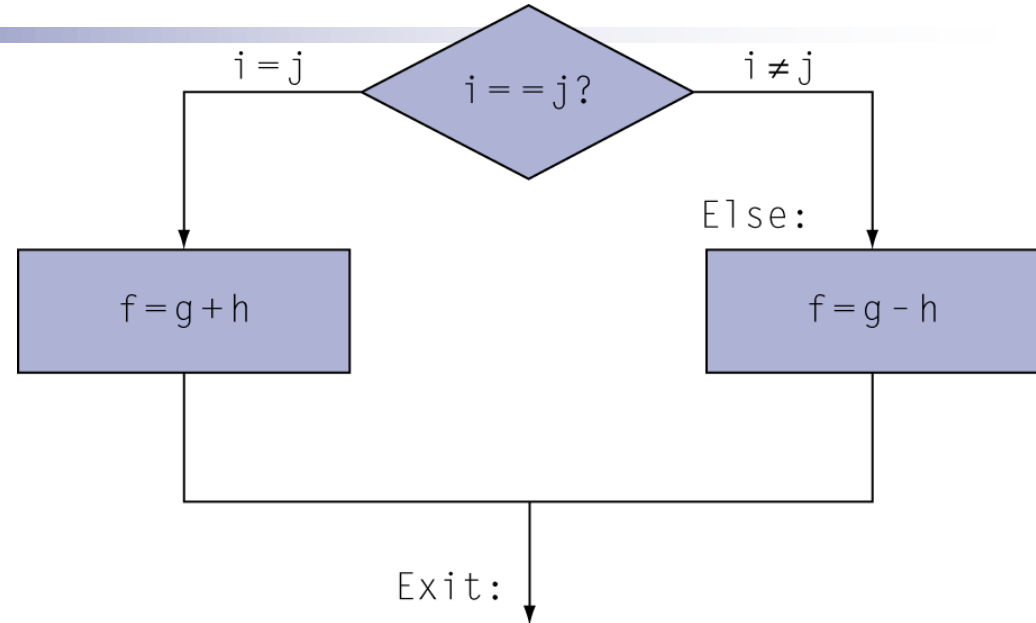
- What distinguishes a computer from a simple calculator is its ability to make decisions.
- Branch to a labeled instruction if a condition is true
 - Otherwise, continue sequentially
- `beq rs1, rs2, L1`
- `bne rs1, rs2, L1`

Compiling If Statements

- Example

C code:

```
if (i==j)
    f = g+h;
else
    f = g-h;
```



- `f, g, h, i, j` in `x19, x20, x21, x22, x23`
- Compiled RISC-V code?
- HLL better than Assembly as no labels/branches used
- Assembler relieves the compiler/assembly programmer from the tedious task of calculating branch addresses

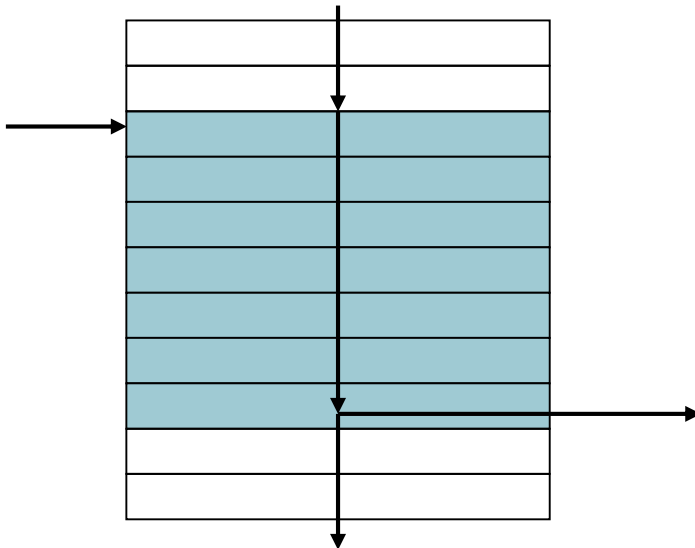
Compiling Loop Statements

- C code:

```
while (save[i] == k)  
    i += 1;
```
- i in x22, k in x24, address of save in x25
- Compiled RISC-V code?

Basic Blocks

- A **basic block** is a sequence of instructions with
 - No embedded branches (except at end)
 - No branch targets (except at beginning)



- A **compiler identifies basic blocks for optimization**
- An advanced processor can accelerate execution of basic blocks

More Conditional Operations

- `blt rs1, rs2, L1`
- `bge rs1, rs2, L1`
- Example
 - `if (a > b)`
 `a += 1; // a in x22, b in x23`
 - Compiled RISC-V code ?

Signed vs. Unsigned

- Comparison tests must take into consideration whether the numbers compared are signed or unsigned
- Signed comparison: blt, bge
- Unsigned comparison: bltu, bgeu
- Example
 - x22 = 1111 1111 1111 1111 1111 1111 1111 1111
 - x23 = 0000 0000 0000 0000 0000 0000 0000 0001

Chapter Outline

- **2 Instructions: Language of the Computer**
 - 2.1 Introduction
 - 2.2 Operations of the Computer Hardware
 - 2.3 Operands of the Computer Hardware
 - 2.4 Signed and Unsigned Numbers
 - 2.5 Representing Instructions in the Computer
 - 2.6 Logical Operations
 - 2.7 Instructions for Making Decisions
 - **2.8 Supporting Procedures in Computer Hardware**
 - 2.9 Communicating with People
 - 2.10 RISC-V Addressing for Wide Immediates and Addresses
 - 2.11 Parallelism and Instructions: Synchronization
 - 2.12 Translating and Starting a Program
 - 2.13 A C Sort Example to Put it All Together
 - 2.14 to 2.24

Procedures

- **Procedures** in assembly language is one tool programmers use to structure programs for
 - Ease of understanding
 - Code re-reuse
- **Similar to *functions/methods* in high level languages** like C, Python
- **Parameters** act as an interface between the procedure and the rest of the program
- **Spy analogy – cover the tracks!**

Procedure Calling

- Following steps are followed when calling procedures:
 1. Place parameters in registers (x10 to x17)
 2. Transfer control to procedure / callee
 3. Acquire storage for procedure
 4. Perform procedure's operations
 5. Place result in register (x10 to x17) for caller
 6. Return to place of call (address in x1)

Procedure Call Instructions

- Some dedicated RISC-V instructions for procedures
- To call a procedure: **jump and link (jal)**
`jal x1, ProcedureLabel`
 - Address of following instruction put in x1
 - Jumps to target address
- To return from a Procedure: **jump and link register (jalr)**
`jalr x0, 0(x1)`
 - Return address?
- **Program counter (PC)**: A register to store the address of current instruction being executed
 - Value in x1 wrt PC

Using more Registers

- If a compiler needs more registers for a procedure than the eight argument registers, portion of memory/RAM used – termed **spilling** to the memory!
- A special portion of memory, known as **Stack**, is used for such purpose:
 - Stack works as a **last-in-first-out** queue.
 - Stack Pointer (**register x2**)
 - **Push** – place data on stack
 - **Pop** – Remove data from stack
 - Grows downwards!

Leaf Procedure Example

- C code:

```
int leaf_example (int g, int h, int i, int j)
{
    int f;
    f = (g + h) - (i + j);
    return f;
}
```

- Arguments g, ..., j in x10, ..., x13
- f in x20
- temporaries x5, x6
- Need to save x5, x6, x20 on stack

- RISC-V Code